

Creating a Simple Mobile Screen

A Comprehensive Hands-On Guide to Core Flutter Layout & Display Widgets

1. Introduction to Mobile UI Construction in Flutter

Flutter is Google's portable UI toolkit for building beautiful, natively compiled applications for mobile, web, and desktop from a single codebase. At the heart of Flutter's architecture is a fundamental principle: **"Everything is a Widget."** In Flutter, user interface components, layout structures, user interaction handlers, and even decorative styling properties are represented as widgets.

When building a simple mobile screen, developers compose smaller, single-purpose widgets together to form complex structural hierarchies known as *Widget Trees*. Understanding how basic widgets function—and how they interlock—is essential for any mobile software engineer.

Core Concept: Declarative UI

Flutter uses a declarative UI framework. This means you describe what your user interface should look like given the current state of your application, rather than imperatively step-by-step modifying layout components.

2. Key Building Blocks Overview

To craft standard mobile interfaces (such as user profiles, product detail cards, or dashboard screens), you rely on a core set of standard Flutter widgets. Below is a breakdown of the primary widgets used in basic UI development:

1. Text

Displays a string of text with a single style. It allows customization of typography, color, size, weight, alignment, and overflow behavior.

2. Image

Renders visual graphics onto the screen. Flutter supports loading images from local assets, network URLs, files, or memory buffers.

3. Container

A multi-purpose convenience widget combining common positioning, sizing, padding, margins, borders, background colors, and transformations.

4. Button Widgets

Interactive elements such as `ElevatedButton`, `TextButton`, or `OutlinedButton` that execute callback functions upon user touch.

5. Column

A structural layout widget that arranges its child widgets vertically top-to-bottom. Manages vertical alignment and space distribution.

6. Row

A structural layout widget that arranges its child widgets horizontally left-to-right. Essential for multi-column content and horizontal toolbars.

3. Deep Dive into Core Widgets

Container & Visual Decoration

The Container widget is one of Flutter's most versatile visual tools. It wraps around a child element and applies constraints, padding, margins, borders, box shadows, and background colors using the BoxDecoration object.

example_container.dart

```
Container(  
  width: 300.0,  
  padding: const EdgeInsets.all(16.0),  
  margin: const EdgeInsets.symmetric(vertical: 10.0),  
  decoration: BoxDecoration(  
    color: Colors.white,  
    borderRadius: BorderRadius.circular(12.0),  
    boxShadow: [  
      BoxShadow(  
        color: Colors.black.withOpacity(0.05),  
        blurRadius: 8.0,  
        offset: const Offset(0, 4),  
      ),  
    ],  
  ),  
  child: const Text("Container Content"),  
)
```

Layout Alignment: Column & Row

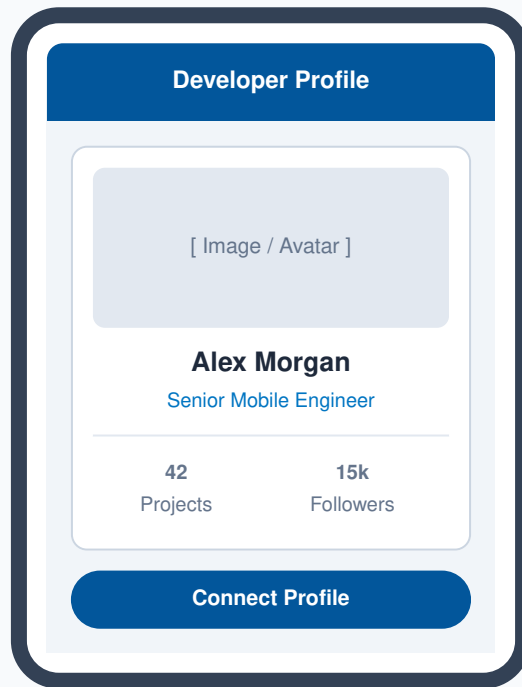
Positioning widgets effectively requires understanding MainAxisAlignment and CrossAxisAlignment:

- **Column:** Main Axis runs vertically (top to bottom); Cross Axis runs horizontally (left to right).
- **Row:** Main Axis runs horizontally (left to right); Cross Axis runs vertically (top to bottom).

Alignment Property	Behavior Description	Typical Use Case
MainAxisAlignment.start	Places children as close to the start of the main axis as possible.	Top-aligned form inputs or left-aligned horizontal rows.
MainAxisAlignment.center	Places children as close to the middle of the main axis as possible.	Centering splash screens or modal dialogues.
MainAxisAlignment.spaceBetween	Distributes free space evenly between adjacent children.	Header bars with title on left and action button on right.
CrossAxisAlignment.stretch	Requires children to fill the cross axis completely.	Full-width button cards inside a vertical column.

4. Designing the Targeted UI: Profile Card Screen

To apply these concepts in a practical context, we will construct a clean, modern user profile card mobile screen. Below is the structural architectural plan and wireframe for our target interface.



5. Complete Code Implementation

Below is the production-ready Dart code leveraging standard Scaffold, Container, Column, Row, Image, Text, and ElevatedButton widgets.

```
main_profile_screen.dart

import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Profile Screen',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: const ProfileScreen(),
    );
  }
}

class ProfileScreen extends StatelessWidget {
```

```

const ProfileScreen({Key? key}) : super(key: key);

@override
Widget build(BuildContext context) {
  return Scaffold(
    backgroundColor: const Color(0xFFFF8FAFC),
    appBar: AppBar(
      title: const Text('Developer Profile'),
      backgroundColor: const Color(0xFF02569B),
      elevation: 0,
      centerTitle: true,
    ),
    body: Center(
      child: SingleChildScrollView(
        child: Padding(
          padding: const EdgeInsets.all(20.0),
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              // --- PROFILE CARD CONTAINER ---
              Container(
                width: double.infinity,
                padding: const EdgeInsets.all(24.0),
                decoration: BoxDecoration(
                  color: Colors.white,
                  borderRadius: BorderRadius.circular(16.0),
                  boxShadow: [
                    BoxShadow(
                      color: Colors.black.withOpacity(0.06),
                      blurRadius: 12.0,
                      offset: const Offset(0, 4),
                    ),
                  ],
                ),
                child: Column(
                  children: [
                    // --- IMAGE WIDGET (AVATAR) ---
                    ClipRect(
                      borderRadius: BorderRadius.circular(50.0),
                      child: Image.network(
                        'https://via.placeholder.com/100',
                        width: 100,
                        height: 100,
                        fit: BoxFit.cover,
                        errorBuilder: (context, error, stackTrace) {
                          return Container(
                            width: 100,
                            height: 100,
                            color: Colors.blue.shade100,
                            child: const Icon(Icons.person, size: 50, color:
Color(0xFF02569B)),
                          );
                        },
                      ),
                    const SizedBox(height: 16),

                    // --- TEXT WIDGETS ---
                    const Text(
                      'Alex Morgan',

```

```

        style: TextStyle(
          fontSize: 20,
          fontWeight: FontWeight.bold,
          color: Color(0xFF0F172A),
        ),
      ),
      const SizedBox(height: 4),
      const Text(
        'Senior Mobile Engineer',
        style: TextStyle(
          fontSize: 14,
          color: Color(0xFF0175C2),
          fontWeight: FontWeight.w500,
        ),
      ),
      const SizedBox(height: 20),
      const Divider(height: 1, color: Color(0xFFE2E8F0)),
      const SizedBox(height: 20),

      // --- ROW WIDGET (STATS SECTION) ---
      Row(
        mainAxisAlignment: MainAxisAlignment.spaceAround,
        children: [
          _buildStatColumn('42', 'Projects'),
          Container(height: 24, width: 1, color: Color(0xFFE2E8F0)),
          _buildStatColumn('15.2k', 'Followers'),
          Container(height: 24, width: 1, color: Color(0xFFE2E8F0)),
          _buildStatColumn('4.9', 'Rating'),
        ],
      ),
    ],
  ),
),
const SizedBox(height: 24),

// --- BUTTON WIDGET ---
SizedBox(
  width: double.infinity,
  height: 50,
  child: ElevatedButton(
    onPressed: () {
      // Action handler
    },
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFF02569B),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(12),
      ),
      elevation: 2,
    ),
    child: const Text(
      'Connect Profile',
      style: TextStyle(
        fontSize: 16,
        fontWeight: FontWeight.bold,
        color: Colors.white,
      ),
    ),
  ),
),
),
),
),
),

```

```

    ],
  ),
),
),
),
);
}

// Helper method for stat columns
static Widget _buildStatColumn(String value, String label) {
  return Column(
    children: [
      Text(
        value,
        style: const TextStyle(
          fontSize: 16,
          fontWeight: FontWeight.bold,
          color: Color(0xFF0F172A),
        ),
      ),
      const SizedBox(height: 2),
      Text(
        label,
        style: const TextStyle(
          fontSize: 12,
          color: Color(0xFF64748B),
        ),
      ),
    ],
  );
}
}

```

6. Best Practices & Optimization Techniques

Building functional UI screens is only the first step. Writing scalable, performant Flutter UI requires adherence to software engineering standards:

- **Use const Constructors:** Mark widgets with the `const` keyword whenever possible. This prevents unnecessary widget rebuilds during state updates, significantly improving frame rates.
- **Prevent Overflow Errors:** When laying out elements inside a `Row` or `Column`, content can easily exceed screen bounds on smaller devices, causing an *"Unbounded Height / Yellow Striped Overflow"* error. Wrap scrollable content inside a `SingleChildScrollView` or use flexible sizing widgets like `Expanded` and `Flexible`.
- **Component Modularity:** Avoid massive `build()` methods. Extract reusable components into small sub-widgets or private helper methods (such as the `_buildStatColumn` method used above).
- **Responsive Spacing:** Utilize `SizedBox` for fixed element spacing instead of empty `Containers` or dynamic paddings.

Key Takeaway

Combining simple primitives—like Text, Image, Container, Row, and Column—provides the computational foundation for over 90% of screen layouts in modern mobile application development.